

IDEXAL · AGENTIC IDE

تقرير مشروع Idexal الشامل

مراجعة معمارية عميقة، جرد كامل لما تم إنجازه وما تبقى، الثغرات المكتشفة، القرارات المعمارية، والخطة المستقبلية لبناء IDE وكيالي منافس ل Claude Code و Codex و Antigravity

إعداد: Idexal — بالتعاون مع Claude (Anthropic)

لصالح: Zakariae Lahbabi

التاريخ: 3 أغسطس 2026

idexal.com · github.com/idexal/idexal



١	1. نظرة عامة وموجز تنفيذي
٢	2. اكتشاف مهم: خريطة مستودعات Idexal الحقيقية
٣	3. الحالة الحالية بالتفصيل — ما اكتمل فعلياً
٤	4. العمل المُنجز في جلسة المراجعة هذه
٥	5. الأخطاء والثغرات المكتشفة
٦	6. القرارات المعمارية (ADRs)
٧	7. ما تبقى — مصفوفة كاملة حسب المراحل
٨	8. الاقتراحات
٩	9. أفكار مستقبلية
١٠	10. الخطوات التالية الفورية

١ نظرة عامة وموجز تنفيذي

Idexal مشروع طموح لبناء **Agentic IDE** كامل ومتكامل — بيئة تطوير وكيلية حرة، متعددة المزودين، بدون قيود — تتنافس Claude Code وCodex وAntigravity. الأساس التقني هو فرع كامل من مستودع VS Code (Code OSS) مفتوح المصدر من مايكروسوفت، مُعاد بناؤه وتطويره تحت هوية Idexal، مع نواة ذكاء اصطناعي مستقلة مبنية خصيصاً (`idexal/ai-core`) وبوابة سحابية مجانية (`idexal/cloud-provider`)، ودومين مخصص `idexal.com`.

الخلاصة الأهم: هذا ليس مشروعاً نظرياً في مرحلة الأفكار. بعد مراجعة معمارية عميقة لكامل القاعدة البرمجية، تبين أن نواة الذكاء الاصطناعي (`idexal/ai-core`) هي **كود حقيقي يعمل ومُختبر بالكامل** — نحو 11,400 سطر Node.js خالص بدون أي تبعية خارجية، مع 34 اختباراً تكاملياً حقيقياً ينفذ فعلياً: مزودين متعددين، Fallback تلقائي، وكلاء متعددين (مخطط ← منفذ ← مراجع)، وذاكرة SQLite طويلة المدى مع استرجاع دلالي (Embeddings).

الأهداف الاستراتيجية

- محرك كامل الميزات (تعديل، تصحيح أخطاء، Git، طرفية، إضافات) — موروث من VS Code.
- وكلاء ذكاء اصطناعي متعددون يتعاونون على المهام (تخطيط ← تنفيذ ← مراجعة).
- دعم متعدد المزودين بالكامل (Anthropic, OpenAI، وأي مزود متوافق) مع Fallback تلقائي عند الفشل.
- ذاكرة طويلة المدى للوكلاء عبر الجلسات (SQLite محلي، حقائق/قرارات/تفضيلات).
- CLI وطرفية خاصة باسم `idexal` تعمل من أي مكان مثل `claude` و `codex`.
- حرية كاملة: صفر قيود، صفر اشتراك إلزامي، مصدر مفتوح (MIT).

المنافسون المرجعيون

المنتج	الجهة	ما نستلهمه منه
Claude Code	Anthropic	تجربة CLI/وكيل نظيفة، ذاكرة طويلة المدى، Subagents، أدوات ملفات/طرفية آمنة
Codex	OpenAI	وضع تنفيذ غير تفاعلي (exec)، تكامل CLI سلس
Antigravity	Google	واجهة "قمرة قيادة" متعددة الوكلاء، تتبع مهام مرئي

٢

اكتشاف مهم: خريطة مستودعات Idexal الحقيقية

أثناء إعداد التخزين الاحترافي (Git) لهذا المشروع، تبين وجود عدة مستودعات فعلية أخرى تحت حساب `github.com/idexal` لم تكن معروفة عند بدء المراجعة. هذا يعني أن "Idexal" كان قيد التطوير على أكثر من جبهة، ويجب توثيق هذا بدقة لتفادي الازدواجية أو فقدان العمل.

المستودع	الرؤية	الوصف الفعلي	الحالة بعد هذه الجلسة
<code>idexal/code</code>	خاص	مشروع مختلف كلياً عن فرع VS Code — عميل وكيل برمجي (CLI/TUI) مبني بلغة/بيئة Bun، وصل الإصدار v1.5.0 بتطوير نشط جداً حتى 25 يونيو 2026: أساس Fallback بين النماذج، ملف تعريف تلقائي لكل مشروع (project profile)، ثماني مهارات (skills) مدمجة، رسم بياني معرفي (knowledge graph)، تكامل Crawl4AI و Markdown، لاستيعاب المعرفة الخارجية.	أرشف (Archived) بقرار المستخدم — محفوظ بالكامل وقابل للرجوع إليه، لكنه لم يعد المسار النشط.
<code>idexal/idexal-code</code>	عام	مستودع تسويقي/توزيع جاهز لمنتج "Idexal Code" — يحتوي شعاراً ولقطة شاشة وسكرت تثبيت، ويصفه كـ"تجربة وكيل برمجي احترافي على سطح المكتب: تعدد نماذج، Fallback تلقائي، مهام متعددة الوكلاء، إكمال تلقائي مضمّن، واجهة داكنة أنيقة".	لم يلمس — يحتاج قراراً لاحقاً حول محاذاته مع الاتجاه الجديد.
<code>idexal/platform</code>	خاص	SaaS حقيقي مبني بـ Next.js + Drizzle ORM لبوابة ذكاء اصطناعي مدفوعة حسب الاستخدام (<code>api.idexal.com</code>) — أكثر نضجاً بكثير من <code>idexal/cloud-provider</code> المحلي.	لم يلمس — يجب تقييمه قبل استكمال تطوير <code>idexal/cloud-provider</code> لتفادي بناء نفس الشيء مرتين (راجع الاقتراحات، 8\$).
<code>idexal/idexalwork</code> , <code>idexal/web-*</code> , <code>idexal/website</code> , <code>idexal/skills</code>	متنوع	مواقع وأدوات مساعدة أخرى (موقع <code>idexal.com</code> ، أدوات CLI فرعية، مهارات).	لم تُفحص بعمق — خارج نطاق هذه المراجعة.
<code>idexal/idexal</code>	خاص (جديد)	المستودع الرسمي الجديد لفرع VS Code — تم إنشاؤه ودفع الحالة الأساسية إليه في هذه الجلسة.	نشط — هذا هو المسار المعتمد للمضي قدماً.

توصية: `idexal/code` المؤرشف يحتوي ميزات مُختبرة وحقيقية (Fallback)، ملف تعريف المشروع التلقائي، المهارات، الرسم البياني المعرفي) تستحق المراجعة كمصدر إلهام هندسي عند بناء ميزات مشابهة في `idexal/ai-core` — لا تُهدر هذه الخبرة الهندسية السابقة فقط لأن المسار تغيّر. راجع الاقتراح المفصّل في 8\$.

3.1 الهوية والعلامة التجارية

العنصر	الحالة	ملاحظة
product.json (الاسم، urlProtocol، applicationName، مجلدات البيانات، GUIDs)	مكتمل	—
package.json (الاسم، المستودع، author)	مكتمل	author مُحدَّث من Microsoft إلى Idexal اليوم
أيقونات win32/darwin/linux	مكتمل اليوم	كانت جاهزة في مجلد منفصل ولم تكن مُطبَّقة — طُبِّقت اليوم
licenseUrl / serverLicenseUrl	مكتمل اليوم	كانت تشير لمستودع مايكروسوفت
webviewControllerExternalBaseUrlTemplate	دين تقني	لا يزال يشير لـ vscode-cdn.net — يحتاج استضافة CDN خاصة لاحقاً، غير حرج للتنشغيل المحلي
voiceWsUrl	دين تقني	يشير لخادم صوت داخلي بمايكروسوفت — يحتاج تعطيل الميزة أو بديل
أيقونة حزمة RPM (Linux، صيغة XPM)	مؤجِّل	يحتاج تحويل صورة، غير حرج لتوزيع Windows الأولي

Idexal AI Core 3.2 — النواة (idexal/ai-core)

~11,400 سطر Node.js، صفر تبعيات خارجية (يستخدم `node:sqlite` و `fetch` المدمجين في 34، Node 24) اختباراً تكاملياً حقيقياً (وليس 6 كما كانت التوثيقات القديمة تذكر — صُحِّح اليوم).

المزودون (Providers)

- **Anthropic**: تطبيق أصلي كامل لـ Messages API، بث SSE حقيقي (text_delta، input_json_delta، message_delta)، استدعاء أدوات في الاتجاهين.
- **OpenAI-compatible**: عميل واحد يخدم OpenAI و OpenRouter و Groq و DeepSeek و Mistral و Together و Ollama و LM Studio وأي endpoint مخصص، عبر جدول `baseUrl` فقط — تصميم صادق وليس ادعاءً تسويقياً.
- بث كامل (Streaming) لكل المزودين، تجميع استدعاءات الأدوات عبر أجزاء SSE متعددة، Embeddings حقيقية عبر `/embeddings`.

محرك Fallback

ترتيب حسب الأولوية، تجاهل المزودين في فترة cooldown أو بدون مفاتيح، تراجع أُسِّي (backoff) عند الفشل، بث أحداث `provider/error/done` للواجهة. مُختبر بما في ذلك فشل منتصف البث والتبديل التلقائي.

الوكلاء المتعددون (Multi-Agent)

خط أنابيب حقيقي: مخطط (Planner) — منفذون (Executors) لكل خطوة (كل منها ChatLoop مستقل بسجل رسائل خاص) ← مراجع (Reviewer) اختياري يمكنه طلب إعادة تنفيذ مرة واحدة. أداة `use_subagent` تسمح بتفويض مهمة لوكيل فرعي متوافق مع نمط Claude Code `((. Claude/agents/*.md`.

دقة مهمة: التنفيذ حالياً تسلسلي (خطوة بعد خطوة) وليس متوازيًا. إعداد `maxParallelAgents` موجود في الإعدادات لكنه غير مُستخدم فعلياً في محرك التنفيذ — تم تصحيح هذا في التوثيق اليوم (كان يُوحي بخلاف الواقع).

الذاكرة طويلة المدى

SQLite محلي دائم (node:sqlite) بخمسة أنواع ذاكرة: جلسة، حقيقة، حلقة، قرار، تفضيل. استرجاع لغوي (Keyword) + استرجاع دلالي حقيقي عبر Embeddings مع تخزين مؤقت للمتجهات، وبتراجع تلقائياً للاسترجاع اللغوي عند فشل المزود. حقن تلقائي في System Prompt قبل كل مهمة.

أدوات الوكيل والحماية

11 أداة: قراءة/كتابة/سرد ملفات، بحث، حالة Git، أوامر طرفية (خلفية وتفاعلية). حماية تجاوز المسار (path-escape) عبر فحص حدود صارم يمنع هجوم "البادنة الشقيقة" (مثل مطابقة `proj-evil/` بالخطأ مع جذر `proj`) — مُختبرة وتنتج. لا يوجد قيد على أوامر الطرفية نفسها (بنفس نموذج ثقة أداة Bash في Claude Code) — قرار تصميم متعمد، ليس ثغرة سهو.

واجهة الأوامر (CLI)

— `idexal-ai run / stream / agent / providers / memory / config / usage / add-provider / setup / compat migrate`
كلها مُطبَّقة فعلياً وتعمل، مع اختبارات end-to-end حقيقية تُشغّل الـ CLI كعملية فرعية وتراقب الإدخال/الإخراج.

Idexal Cloud Provider 3.3 — البوابة السحابية المجانية

- بوابة HTTP متوافقة مع Node.js، (`idexal/cloud-provider`)، OpenAI خالص بدون تبعيات، تعمل كخيار افتراضي بلا إعدادات (`idexal-cloud` في قائمة المزودين). 11 اختباراً تكاملياً (بعد التصحيح اليوم من 8 المذكورة سابقاً).
- مترجم بروتوكول OpenAI ⇌ Anthropic كامل ثنائي الاتجاه (بما في ذلك تدفق استدعاء الأدوات كدلتا تراكمية — نقطة دقيقة تم اختبارها).
 - نظام حصص Freemium: عدادات يومية SQLite لكل هوية (مجهول/مفتاح مسجّل)، محدّد burst بمعدل منزلق.
 - توجيه مع Fallback عبر المنابع (Anthropic ← OpenAI ← Ollama افتراضياً).
 - **غير منشورة بعد** — `api.idexal.com` ليس لديه استضافة فعلية بعد. يُنصح بمراجعة `idexal/platform` الأكثر نضجاً قبل المتابعة (انظر 8§).

3.4 حالة البناء والتشغيل

العنصر	الحالة
(devDependency) Electron 42.7.1، موارد win32/darwin/linux موجودة	جاهز بنويًا
سكريبتات التشغيل <code>idexal.sh</code> و <code>scripts/idexal.bat</code>	صحيحة ومكتملة
<code>npm install</code> كامل	قيد التنفيذ الآن
أول تجميع (build-fast) وتشغيل مرئي	التالي مباشرة بعد هذا التقرير
Rust toolchain (لبناء CLI أصلي <code>idexal</code> في <code>/cli</code>)	غير مثبت — يحتاج OpenSSL + <code>rustup</code> عبر <code>vcpkg</code> على ويندوز
Visual Studio Build Tools 2022 (MSVC)	مثبت مسبقاً — يكفي لوحدة Node/Electron الأصلية

Rust CLI 3.5 بلغة (cli/)

كود حقيقي وليس هيكلاً فارغاً: `commands/agent.rs` (عميل AHP عبر WebSocket/نفق مطوّرين + مصادقة device-flow)، `commands/agent_host.rs` (784 سطرًا — خادم/مُشرف حقيقي يدير عملية فرعية، lockfile، إعادة تشغيل عند التحديث). يعتمد على حزمتي

`ahp` و `ahp-types` المنشورتين فعلياً على crates.io. غير قابل للبناء حالياً فقط لغياب Rust toolchain على الجهاز — لا عائق هندسي آخر.

3.6 امتداد GitHub Copilot Chat الموروث (extensions/copilot)

641 ألف سطر — كود إنتاجي حقيقي من GitHub/مايكروسوفت. حلقات الوكيل الرئيسية (، `executionSubagentToolCallingLoop.ts` `searchSubagentToolCallingLoop.ts`، مبنية على محرك `toolCallingLoop.ts` الضخم 2,431 سطرًا) مرتبطة افتراضياً بخادم GitHub الخاص (CAPI) وتتطلب اشتراك GitHub Copilot فعلياً — ما يتعارض مع رؤية Idexal "الحرّة بدون قيود". **الاستثناء المهم:** طبقة BYOK (~7,200 سطر) تتصل مباشرة بـ Anthropic/OpenAI/Gemini/Ollama بدون أي حساب GitHub، وتُستخدم كمرجع تصميم مفيد. القرار المعماري المُعتمد: عدم البناء فوق حلقات CAPI، والاعتماد حصراً على `ai-core` (راجع ADR 0001 في §6).

3.7 نظام Agent Host الأصلي في (AHP) VS Code

1,286 — `src/vs/platform/agentHost/` ملفاً، 247 ألف سطر. بروتوكول حقيقي وموثّق بإصدار (AHP) عبر WebSocket + lockfile، مصمّم أصلاً لاستضافة وكلاء خارجيين متعددين (Claude Agent SDK، Codex CLI، GitHub Copilot CLI) كخلفيات قابلة للتوصيل. هذا هو الهدف النهائي لدمج `ai-core` كخلفية أصلية في واجهة الوكلاء بالمحرر — لكنه مؤجّل حتى تستقر لوحة أبسط أولاً (راجع ADR 0002 في §6).

1. مراجعة معمارية عميقة لكامل القاعدة البرمجية عبر ثلاثة تحقيقات متوازية (ai-core/cloud-provider, حالة البناء والعلامة التجارية, امتداد Copilot وAHP).
2. تهيئة مستودع Git رسمي لأول مرة للمشروع + `.gitignore`. مخصص ل Idexal.
3. إنشاء `CHANGELOG.md` و `docs/adr/` — بدءاً بقرارين معماريين موثّقين (§6).
4. إصلاح خطأ حقيقي: حد ال burst rate limit في cloud-provider لم يكن يُطَبَّق حسب فئة العميل — العملاء المسجّلون كانوا يُقَيّدون بنفس حد المجهولين. تم الإصلاح + إضافة اختبار انحدار (regression test).
5. توثيق ثغرة تزييف الهوية المجانية (device header / X-Forwarded-For غير مُتَحَقَّق منهما) كبند مانع نشر صريح في DESIGN.md.
6. تصحيح أرقام الاختبارات القديمة في ROADMAP.md و DESIGN.md (34→33 وفعلياً بعد الإصلاح، و8→11).
7. توضيح أن تنفيذ الوكلاء المتعددين تسلسلي حالياً وليس متوازيّاً (كان التوثيق مُضِلّاً).
8. تطبيق أيقونات Idexal الجديدة على win32/darwin/linux, وتحديث روابط الترخيص في product.json و author و package.json.
9. اكتشاف مستودعات Idexal الحقيقية الأخرى على (§2) GitHub — قرار حاسم مع المستخدم حول العلاقة بينها.
10. أرشفة `idexal/code` القديم بأمان (قابل للتراجع, لم يُحذف).
11. إنشاء مستودع `idexal/idexal` الخاص الجديد ودفع أول commit أساس إليه بنجاح (17,236 ملفاً).
12. حل مشكلة Git LFS لملفات اختبار مؤقتة (97 ملف sqlite لذاكرة تخزين اختبارات المحاكاة) عبر استثنائها من التتبع بدل تعقيد إعداد LFS لبيانات غير جوهرية.
13. تشغيل `npm install` الكامل (كان تثبيثاً جزئياً غير مكتمل).

٥

الأخطاء والثغرات المكتشفة

#	الوصف	الموقع	الخطورة	الحالة
1	حد burst rate limit لا يُطَبَّق حسب الفئة (مفتاح مقابل مجهول) — متغير محسوب وغير مُستخدم	cloud-provider/src/server.ts + quota.ts	متوسطة	أُصلح اليوم + اختبار
2	تزييف الهوية المجانية ممكن عبر ترويسة جهاز عشوائية أو X-Forwarded-For غير موثَّق — يتيح تجاوز الحصة اليومية	cloud-provider/src/auth.ts	عالية (قبل النشر العام)	مُوثَّقة كمانع نشر — غير مُصلحة (البوابة غير منشورة فعلياً بعد)
3	maxParallelAgents إعداد ميت — التنفيذ الفعلي تسلسلي رغم الاسم	ai-core/src/agents/orchestrator.ts	منخفضة (توثيق مُضِلّ، ليس عطلاً وظيفياً)	التوثيق صَحَّح اليوم — التنفيذ الفعلي للتوازي بند مستقبلي
4	ملف قاعدة بيانات فعلي (gateway-quota.db) كان في شجرة العمل بدون .gitignore	cloud-provider/	منخفضة	أُصلح اليوم
5	ProviderHealth.consecutiveFailures يُتَتَبَع لكن لا يؤثر على مدة الـ cooldown (ثابتة 30 ثانية دوماً)	ai-core/src/providers/registry.ts	منخفضة	لم تُعالج — بند مستقبلي (9\$)
6	دالة assertConfigured مُصدّرة لكن غير مستخدمة في أي مكان	ai-core/src/providers/factory.ts	تافهة	لم تُعالج
7	لا يوجد فحص أنواع تلقائي (tsc --noEmit) في مسار الاختبار/التشغيل لأي من ai-core أو cloud-provider	ai-core, cloud-provider	متوسطة	لم تُعالج — سمحت للخطأ #1 بالمرور دون اكتشاف

ADR 0001 — استخدام Idexal AI Core بدل حلقات Copilot CAPI

نواة الوكلاء الرسمية هي `idexal/ai-core` ، وليس حلقات `extensions/copilot` المرتبطة بخادم GitHub الخاص واشتراك Copilot. الامتداد يبقى كمرجع لإعادة استخدام مكونات واجهة محددة فقط.

ADR 0002 — لوحة Idexal خفيفة أولاً، ثم AHP الكامل لاحقاً

بدل الاستثمار المباشر في تكامل AHP/agentHost الضخم (247 ألف سطر، يتطلب Rust)، نبني أولاً امتداد/لوحة بسيطة تستدعي `ai-` مباشرة كعملية Node، لنحصل على نتيجة مرئية واختبار حقيقي سريعاً. التكامل الكامل مع AHP يبقى هدفاً استراتيجياً موثقاً للمرحلة 3/4.

المرحلة 0 — التأسيس والهوية

مكتملة تقريباً — تبقى فقط: CDN خاص للـ webview، بديل لميزة الصوت، أيقونة RPM.

المرحلة 1 — CLI خاصة بنا (idexal)

- تثبيت OpenSSL + Rust toolchain (rustup) عبر vcpkg على ويندوز.
- بناء الـ CLI من `cli/` وربطه بأمر `idexal` في `PATH`.
- `idexal run/agent/tunnel/serve-web` بهوية Idexal الكاملة.

المرحلة 2 — الطرفية الذكية

- تفعيل طرفية `xterm.js` بهوية Idexal.
- لوحة "Agent Terminal" مع تمييز أوامر الوكيل وتدفق موافقة قبل الأوامر الخطيرة.

المرحلة 3 — تعدد الوكلاء الحقيقي

- لوحة "Idexal Agent" جديدة في المحرر تستدعي `ai-core` (وضع `--json-stream`).
- تفعيل تنفيذ متواز حقيقي (إصلاح `maxParallelAgents` الميت).
- مدير سياق مشترك بين الوكلاء (`Workspace Recorder + Session Store`).

المرحلة 4 — دمج الوكلاء في المحرر

- `Diff` مرئي متدفق لتعديلات الوكيل + أزرار قبول/تراجع.
- لوحة نشاط تعرض حالة كل وكيل لحظياً.

المرحلة 5 — الحسابات والمزودون

- تخزين آمن لمفاتيح API محلياً عبر واجهة المحرر.
- **حسم العلاقة مع `idexal/platform` (SaaS جاهز أكثر نضجاً — راجع 8§)** قبل استكمال `cloud-provider`.
- إغلاق ثغرة تزييف الهوية المجانية (الخطأ #2 أعلاه) — مانع نشر صريح.

المرحلة 6 — التوزيع والنشر

- حزم `Windows (Inno Setup) / macOS (dmg) / Linux (deb/rpm/snap)`.
- موقع `idexal.com` كامل (تحميل، توثيق، تسويق) — قد يُبنى على `idexal/website` الموجود بالفعل.
- تحديث تلقائي (البنية موجودة جزئياً في `cli/src/update_service.rs`).

8.1 حسم ازدواجية cloud-provider مقابل platform (أولوية عالية)

`idexal/platform` مشروع Next.js + Drizzle حقيقي مُصمّم خصيصاً كـ "usage-metered AI gateway SaaS" لنفس النطاقات `platform.idexal.com` و `api.idexal.com` التي يستهدفها `idexal/cloud-provider` المحلي. الاستمرار في تطوير الاثنين بالتوازي يعني ازدواج جهد حقيقي. يُنصح بجلسة تقييم مخصصة (مراجعة كود `idexal/platform` بنفس عمق هذه المراجعة) لتحديد: هل نهجر `cloud-provider` لصالح `platform` الأنصح؟ أم يبقى `cloud-provider` كطبقة `gateway` بسيطة خفيفة الوزن (Node خالص بدون تبعيات) و `platform` كطبقة `SaaS` فويرة فوقه؟

8.2 استلهم ميزات `idexal/code` المؤرشف قبل إعادة اختراعها

المشروع المؤرشف يحوي حلولاً مُختبرة وحقيقية لمشاكل نحتاجها بالضبط: أساس Fallback بين النماذج (مشابه لكن مستقل عمّا في `ai-core`)، اكتشاف تلقائي لملف تعريف كل مشروع (`project profile` عبر 11 نوع ملف تعريف)، نظام مهارات (`Skills`) قابل للتوسعة، ورسم بياني معرفي (`Knowledge Graph`) مبني على `Crawl4AI` و `MarkItDown`. قبل بناء أي من هذه الميزات من الصفر في `ai-core` لاحقاً، راجع التطبيق الموجود في `idexal/code` كمرجع تصميم مُثبّت الجدوى.

8.3 إعداد فحص أنواع تلقائي (CI أساسي)

غياب `tsc --noEmit` في مسار الاختبار سمح بمرور خطأ حقيقي (متغير محسوب وغير مُستخدم في `enforceBurst`) دون اكتشاف. حتى قبل بناء CI كامل على `GitHub Actions`، إضافة خطوة فحص أنواع بسيطة لكل من `ai-core` و `cloud-provider` تمنع فئة كاملة من الأخطاء المشابهة.

8.4 إغلاق ثغرة الحصة المجانية قبل أي نشر علني للبوابة

كما وُثّق في §5 (الخطأ #2) — بند مانع نشر، ليس تحسیناً اختيارياً.

8.5 الحفاظ على سجل Git تصاعدي مع VS Code الأصلي

المستودع الحالي بدأ بتاريخ Git جديد كلياً (`commit` واحد يحتوي كل شيء) بدل الحفاظ على تاريخ مايكروسوفت الأصلي. هذا مقبول عملياً وأسرع، لكنه يعني صعوبة أكبر لاحقاً في سحب تحديثات أمنية من VS Code الأصلي عبر `git rebase/merge` نظيف. يُنصح على الأقل بإضافة `remote upstream` يشير إلى `github.com/microsoft/vscode` (دون دمج فوري) لإبقاء الخيار مفتوحاً لاحقاً.

8.6 قرار الترخيص والعلنية

الرؤية المعلنة "المصدر مفتوح (MIT)" لكن المستودع الرئيسي الآن خاص (`Private`) بقرار حكيم للمرحلة المبكرة. يُنصح بتحديد معيار واضح لتوقيت التحول إلى `Public` (مثلاً: بعد أول تشغيل مرئي ناجح + إغلاق ثغرات الأمان المعروفة + تنظيف أي أسرار من سجل `Git`) بدل تركه مفتوحاً بلا معيار.

تقنياً

منتجياً وتجارياً

- تنفيذ متوازن حقيقي للوكلاء المستقلين (Promise.all) حيث لا تعتمد الخطوات على بعضها).
- cooldown تصاعدي حسب consecutiveFailures بدل 30 ثانية ثابتة دوماً.
- تكامل AHP كامل — Idexal كخلفية أصلية بجانب Claude/Codex في واجهة الوكلاء بالمحرر.
- سوق إضافات (Marketplace) خاص بـ Idexal أو تكامل مع الحالي.
- وضع "Ask/Explore/Plan" مخصص بهوية Idexal (مستوحى من أنماط agents/vscode-node الموجودة).
- دعم رؤية (Vision) للنماذج متعددة الوسائط في الوكلاء.
- نموذج تسعير واضح لـ Idexal Cloud Gateway (طبقة مجانية + مفتاح مدفوع).
- موقع idexal.com بتوثيق شامل (عربي/إنجليزي) وقصص استخدام.
- سياسة خصوصية وتبليغ اختياري بالكامل (لا تبليغ إلزامية — يتمشى مع "الحرية").
- مجتمع مطورين (Discord/GitHub Discussions) بمجرد فتح المصدر.
- مراجعة أمنية مستقلة (Security Review) قبل أي Beta عام.
- دعم لغات توطين إضافية للواجهة تتجاوز العربية/الإنجليزية.

1. إتمام `npm install` الكامل (قيد التنفيذ الآن).
2. `npm run build-fast` لأول تجميع سريع.
3. تشغيل `scripts\idexal.bat` والتحقق البصري من الإقلاع بهوية Idexal الجديدة (أيقونة، اسم) بدون أخطاء.
4. تصميم ثلاثة مخططات (Mockups) لهيكل الواجهة الجديدة لاختيار المستخدم بينها.
5. بعد الاعتماد: بناء لوحة "Idexal Agent" الأولى المتصلة بـ ai-core مباشرة.
6. جلسة تقييم مخصصة لعلاقة cloud-provider بـ idexal/platform (\$8.1) — أولوية موازية.

تقرير مُؤَدَّ آلياً استناداً إلى مراجعة معمارية حقيقية للكود المصدري بتاريخ 03-08-2026. جميع الأرقام (عدد الأسطر، الاختبارات، الملفات) مُستخرجة مباشرة من القاعدة البرمجية وليست تقديرية. يُحفظ هذا التقرير في `docs/reports/` ضمن المستودع لتتبع تطور الحالة عبر الزمن.